Java — Zero to Master

24 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction, JDK and First Program
02 Variables and Data Types
03 Operators and Type Casting
04 Control Flow
05 Methods
06 Arrays and Strings
07 Classes and Objects

Intermediate (7 topics)

08 OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
09 static, final, this, super
10 Exception Handling
11 Collections Framework
12 Generics
13 File I/O and NIO
14 Packages and Access Modifiers

Advanced (6 topics)

15 Lambdas and Functional Interfaces
16 Streams API and Optional
17 Multithreading and Concurrency
18 Records, Sealed Classes, Pattern Matching
19 JDBC and Databases
20 Maven/Gradle and Testing

Master (4 topics)

21	JVM Internals and Garbage Collection
22	Design Patterns and SOLID
23	Real Projects
24	Best Practices and Interview Prep

Beginner Level

01. Introduction, JDK and First Program

DEFINITION

Java is a statically typed, object-oriented language that compiles to bytecode and runs on the JVM ('write once, run anywhere'). Install a JDK, then compile with `javac` and run with `java`.

SYNTAX

```
javac Hello.java
java Hello
```

EXAMPLE

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A Tamil film can be dubbed and shown in any theatre; Java bytecode runs on any machine with a JVM.

தமிழ்

ஒரு தமிழ்ப் படம் எந்தத் திரையரங்கிலும் டப்பிங் செய்து காட்டப்படுவது போல Java bytecode JVM உள்ள எந்த கணினியிலும் இயங்கும்.

PROS

- Platform independent
- Strong ecosystem
- Mature, secure and fast JVM

CONS

- Verbose syntax
- Higher memory use
- Slower startup than native languages

ADVANCED / NEW FEATURES

- Java 25 LTS: compact source files and instance `main`, scoped values
- Virtual threads (21)
- Records, sealed classes, pattern matching

02. Variables and Data Types

DEFINITION

Primitives: `byte short int long float double char boolean`. `String` is a class. Use `var` for local type inference.

SYNTAX

```
type name = value;
var x = 10;
```

EXAMPLE

```
int age = 25;
double price = 99.5;
String name = "Arun";
var count = 10;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Containers in a kitchen: a litre jar for oil, a kilo box for rice. Each type holds only its kind of value.

தமிழ்

சமையலறையில் எண்ணெய்க்கு லிட்டர் ஜாடி, அரிசிக்கு கிலோ டப்பா போல ஒவ்வொரு type-உம் தனக்குரிய மதிப்பை மட்டும் வைத்திருக்கும்.

PROS

- Compile-time type safety
- Clear data sizes

CONS

- Verbose declarations (reduced with `var`)

ADVANCED / NEW FEATURES

- `var` local inference (10)
- Text blocks (15)

03. Operators and Type Casting

DEFINITION

Arithmetic, relational, logical, ternary operators; widening (automatic) and narrowing (explicit cast) conversions.

SYNTAX

```
(type) value
```

EXAMPLE

```
int a = 7 / 2;          // 3
double b = 7 / 2.0;     // 3.5
int c = (int) 9.8;      // 9
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Pouring a litre into a half-litre cup spills; narrowing conversions lose data and need an explicit cast.

தமிழ்

ஒரு லிட்டரை அரை லிட்டர் குவளையில் ஊற்றினால் சிந்தும்; narrowing conversion தரவை இழக்கும் என்பதால் வெளிப்படை cast தேவை.

PROS

- Predictable numeric rules

CONS

- Silent overflow with ints
- Integer division surprises

ADVANCED / NEW FEATURES

- Pattern matching for `instanceof` (16)

04. Control Flow

DEFINITION

`if/else`, `switch` (including switch expressions), `for`, enhanced `for`, `while`, `do-while`, `break`, `continue`.

SYNTAX

```
if (c) { } else { }
switch (x) { case 1 -> ...; }
```

EXAMPLE

```
for (int n : new int[]{1, 2, 3}) {
    System.out.println(n);
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bus conductor checks each passenger's ticket one by one; loops repeat the check for every seat.

தமிழ்

நடத்துநர் ஒவ்வொரு பயணியின் டிக்கெட்டையும் ஒவ்வொன்றாகச் சோதிப்பது போல loops ஒவ்வொரு இருக்கைக்கும் சோதனையை மீண்டும் செய்கின்றன.

PROS

- Simple, readable flow control
- Enhanced `for` is concise

CONS

- Old `switch` fall-through bugs

ADVANCED / NEW FEATURES

- Switch expressions (14)
- Pattern matching in `switch` (21)

05. Methods

DEFINITION

Reusable blocks with parameters and return type; overloading means same name with different parameters.

SYNTAX

```
returnType name(params) { }
```

EXAMPLE

```
static int add(int a, int b) { return a + b; }
static double add(double a, double b) { return a + b; }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A juice machine: input fruit, output juice. Overloading means the same machine name for different fruit types.

தமிழ்

ஜூஸ் இயந்திரம்: பழம் உள்ளே, ஜூஸ் வெளியே; overloading என்பது வெவ்வேறு பழங்களுக்கு ஒரே இயந்திரப் பெயர்.

PROS

- Reusable code
- Overloading improves readability

CONS

- No default parameters
- No standalone functions (must be in a class)

ADVANCED / NEW FEATURES

- Unnamed variables `_` (22)
- Instance main methods (25)

06. Arrays and Strings

DEFINITION

Arrays have fixed size. `String` is immutable; use `StringBuilder` for many concatenations.

SYNTAX

```
int[] a = new int[5];
String s = "text";
```

EXAMPLE

```
int[] marks = {80, 90, 70};
String s = "java";
System.out.println(s.toUpperCase() + marks.length);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A row of lockers with fixed numbers (array); a name board whose letters you cannot change but can replace (String).

தமிழ்

எண்ணிடப்பட்ட நிலையான லாக்கர்கள் (array); எழுத்துகளை மாற்ற முடியாத ஆனால் மாற்றிவிடக்கூடிய பெயர்ப் பலகை (String).

PROS

- Fast indexed access
- String pool saves memory

CONS

- Fixed array size
- Many string concatenations are slow (use StringBuilder)

ADVANCED / NEW FEATURES

- Text blocks
- `String.formatted`, `strip`, `repeat`

07. Classes and Objects

DEFINITION

A class is a blueprint; an object is an instance. Fields hold state, methods hold behaviour, constructors initialise.

SYNTAX

```
class Name { fields; Name() { } void method() { } }
```

EXAMPLE

```
class Student {
    String name;
    Student(String name) { this.name = name; }
    void hello() { System.out.println("Hi " + name); }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A house plan is the class; each house built from it is an object with its own owner and colour.

தமிழ்

வீட்டு வரைபடம் class; அதிலிருந்து கட்டிய ஒவ்வொரு வீடும் தன் உரிமையாளர், நிறத்துடன் ஒரு object.

PROS

- Models real-world things
- Reusable and maintainable

CONS

- Boilerplate for plain data (solved by records)

ADVANCED / NEW FEATURES

- Records (16)
- Sealed classes (17)

Intermediate Level

08. OOP: Encapsulation, Inheritance, Polymorphism, Abstraction

DEFINITION

Hide data with `private` + getters/setters; reuse with `extends`; override methods for polymorphism; model contracts with `abstract` classes and `interface`s.

SYNTAX

```
class B extends A implements I { }
```

EXAMPLE

```
interface Shape { double area(); }
class Circle implements Shape {
    private final double r;
    Circle(double r) { this.r = r; }
    public double area() { return Math.PI * r * r; }
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A family business: the father's skills pass to children (inheritance), each one performs the job differently (polymorphism).

தமிழ்

குடும்ப வணிகம்: தந்தையின் திறமை பிள்ளைகளுக்கு (inheritance), ஒவ்வொருவரும் வேலையை வேறு விதமாகச் செய்வர் (polymorphism).

PROS

- Code reuse
- Extensible design
- Clear contracts

CONS

- Deep hierarchies are rigid
- Single inheritance only

ADVANCED / NEW FEATURES

- Sealed classes/interfaces
- Default and private interface methods

09. static, final, this, super

DEFINITION

`static` belongs to the class; `final` prevents change/override; `this` is the current object; `super` refers to the parent.

SYNTAX

```
static final int MAX = 10;
```

EXAMPLE

```
static final double PI = 3.14159;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A national flag is common to everyone (`static`) and cannot be changed (`final`); `this` is 'me', `super` is 'my parent'.

தமிழ்

தேசியக் கொடி அனைவருக்கும் பொது (`static`), மாற்ற முடியாதது (`final`); `this` 'நான்', `super` 'என் பெற்றோர்'.

PROS

- Constants and shared state
- Immutability aids safety

CONS

- Static state hurts testing

ADVANCED / NEW FEATURES

- Static members in records/interfaces
- Scoped values (25)

10. Exception Handling

DEFINITION

Checked vs unchecked exceptions; `try/catch/finally`, try-with-resources, custom exceptions.

SYNTAX

```
try { } catch (E e) { } finally { }
```

EXAMPLE

```
try (var in = new Scanner(System.in)) {
    int n = Integer.parseInt(in.nextLine());
} catch (NumberFormatException e) {
```

```
    System.out.println("Not a number");
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

If the gas cylinder ends while cooking, you switch to induction instead of abandoning dinner.

தமிழ்

சமைக்கும்போது கேஸ் தீர்ந்தால் இரவு உணவைக் கைவிடாமல் இண்டக்ஷனுக்கு மாறுவது போல.

PROS

- Robust programs
- Checked exceptions document failures

CONS

- Verbose
- Overusing checked exceptions clutters code

ADVANCED / NEW FEATURES

- Try-with-resources improvements
- Helpful NullPointerException messages

11. Collections Framework

DEFINITION

`List` (ArrayList), `Set` (HashSet, TreeSet), `Map` (HashMap), `Queue` / `Deque`; choose by ordering, duplicates and lookup needs.

SYNTAX

```
List<String> l = new ArrayList<>();
```

EXAMPLE

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Arun", 90);
marks.getOrDefault("Ravi", 0);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A school has class lists (List), unique attendance IDs (Set) and roll-number to name registers (Map).

தமிழ்

பள்ளியில் வகுப்புப் பட்டியல் (List), தனித்துவ ID (Set), பதிவெண்-பெயர் பதிவேடு (Map) இருப்பது போல.

PROS

- Rich ready-made data structures
- Consistent API

CONS

- Boxing overhead for primitives
- Choosing wrong type hurts performance

ADVANCED / NEW FEATURES

- Sequenced collections (21)
- `List.of`, `Map.of` immutable factories

12. Generics

DEFINITION

Type-safe containers and methods with type parameters; wildcards `? extends T`.

SYNTAX

```
class Box<T> { T value; }
```

EXAMPLE

```
static <T> T first(List<T> list) { return list.get(0); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A universal tiffin box that carries idli, pongal or rice; generics allow one design for many types.

தமிழ்

இட்லி, பொங்கல், சாதம் எதையும் எடுத்துச் செல்லும் டிபன் பாக்ஸ் போல generics ஒரே வடிவமைப்பை பல வகைகளுக்கு அளிக்கின்றன.

PROS

- Type safety
- No casting

CONS

- Type erasure limits
- Complex wildcards

ADVANCED / NEW FEATURES

- Generic improvements with records/pattern matching

13. File I/O and NIO

DEFINITION

Read/write files with `Files`, `Path`, `BufferedReader`; try-with-resources closes streams.

SYNTAX

```
Files.readAllLines(Path.of("a.txt"))
```

EXAMPLE

```
List<String> lines = Files.readAllLines(Path.of("a.txt"));
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Reading and writing entries in a shop's ledger book; Java I/O reads and writes files and streams.

தமிழ்

கடை கணக்குப் புத்தகத்தில் எழுதுவது/படிப்பது போல Java I/O கோப்புகளைப் படித்து எழுதுகிறது.

PROS

- Powerful NIO API
- Try-with-resources auto closes

CONS

- Many classes (legacy java.io vs NIO)

ADVANCED / NEW FEATURES

- `Files.readString` / `writeString`
- `InputStream.transferTo`

14. Packages and Access Modifiers

DEFINITION

Organise classes in packages; `public`, `protected`, default and `private` control visibility.

SYNTAX

```
package a.b;
import x.Y;
```

EXAMPLE

```java
package com.example.app;
import java.util.List;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Departments in a company each have their own cabinet; packages group classes and access modifiers control who can open them.

தமிழ்

நிறுவனத் துறைகளுக்கு தனி அலமாரி போல packages classes-ஐ தொகுத்து access modifiers யார் திறக்கலாம் என்பதைக் கட்டுப்படுத்துகின்றன.

PROS

- Name collision avoidance
- Encapsulation

CONS

- Long import lists

ADVANCED / NEW FEATURES

- Java Platform Module System (9)
- Module import declarations (25)

Advanced Level

15. Lambdas and Functional Interfaces

DEFINITION

Concise anonymous functions for interfaces with one abstract method (`Runnable`, `Comparator`, `Function`, `Predicate`).

SYNTAX

```
(a, b) -> a + b
```

EXAMPLE

```
list.sort((a, b) -> a.length() - b.length());
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A quick handwritten chit instead of a formal letter; lambdas are short anonymous functions.

தமிழ்

முறையான கடிதத்துக்குப் பதிலாக விரைவான கையெழுத்துச் சீட்டு போல lambdas குறுகிய பெயரில்லா functions.

PROS

- Concise code
- Enables streams

CONS

- Debugging stack traces can be harder

ADVANCED / NEW FEATURES

- Method references
- Unnamed lambda parameters (22)

16. Streams API and Optional

DEFINITION

Declarative pipelines: `filter`, `map`, `collect`, `reduce`; `Optional` models absent values.

SYNTAX

```
list.stream().filter(...).map(...).toList()
```

EXAMPLE

```
List<String> names = users.stream()
    .filter(u -> u.age() > 18).map(User::name).toList();
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A mango processing line: sort ripe ones, pack, count; streams process data in clear steps.

தமிழ்

மாம்பழ பதப்படுத்தும் வரிசை: பழுத்தவற்றைப் பிரி, பேக் செய், எண்ணு; streams தரவை தெளிவான படிகளில் செயலாக்கும்.

PROS

- Declarative and readable
- Easy parallelism

CONS

- Overuse reduces clarity
- Debugging pipelines is harder

ADVANCED / NEW FEATURES

- `Stream.toList()` (16)
- Stream Gatherers (24)

17. Multithreading and Concurrency

DEFINITION

Threads, `ExecutorService`, `CompletableFuture`, locks, `synchronized`, concurrent collections and virtual threads (Java 21).

SYNTAX

```
Thread t = Thread.ofVirtual().start(task);
```

EXAMPLE

```
try (var ex = Executors.newVirtualThreadPerTaskExecutor()) {
    ex.submit(() -> System.out.println("task"));
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

One cashier with many helpers each handling a customer; virtual threads let one JVM handle millions of lightweight tasks.

தமிழ்

ஒரு காசாளருடன் பல உதவியாளர்கள் ஆளுக்கொரு வாடிக்கையாளரைக் கவனிப்பது போல virtual threads ஒரு JVM-இல் மில்லியன் இலகு பணிகளைக் கையாளும்.

PROS

- High throughput
- Simple blocking style
- Better CPU use

CONS

- Race conditions are tricky
- Needs careful shared-state design

ADVANCED / NEW FEATURES

- Virtual threads (21)
- Structured concurrency (preview)
- Scoped values (25)

18. Records, Sealed Classes, Pattern Matching

DEFINITION

Modern Java: `record` for immutable data, `sealed` hierarchies, `switch` pattern matching, text blocks.

SYNTAX

```
record P(int x, int y) { }
sealed interface S permits A, B { }
```

EXAMPLE

```
record Point(int x, int y) {}
String d = switch (shape) { case Circle c -> "circle"; default -> "other"; };
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A visiting card: fixed fields, no editing; records hold immutable data with no boilerplate.

தமிழ்

அடையாள அட்டை: நிலையான புலங்கள், மாற்ற முடியாது; records boilerplate இல்லாமல் மாறாத தரவைத் தாங்குகின்றன.

PROS

- Less boilerplate

- Safer modelling
- Exhaustive switches

CONS

- Records cannot extend classes

ADVANCED / NEW FEATURES

- Record patterns (21)
- Unnamed patterns (22)
- Primitive patterns (preview)

19. JDBC and Databases

DEFINITION

Connect to databases with JDBC; always use `PreparedStatement`; connection pools (HikariCP).

SYNTAX

```
PreparedStatement ps = con.prepareStatement(sql);
```

EXAMPLE

```
try (var ps = con.prepareStatement("SELECT * FROM users WHERE id = ?")) {
    ps.setInt(1, id);
}
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Asking the shop ledger specific questions through a form slip so nobody can tamper with the question.

தமிழ்

கடைப் பதிவேட்டிடம் குறிப்பிட்ட கேள்விகளை படிவச் சீட்டு மூலம் கேட்பது போல; கேள்வியில் யாரும் தலையிட முடியாது.

PROS

- Standard database API
- Prepared statements prevent SQL injection

CONS

- Verbose boilerplate (use JPA/JdbcTemplate)

ADVANCED / NEW FEATURES

- Connection pools like HikariCP
- R2DBC for reactive

20. Maven/Gradle and Testing

DEFINITION

Build tools manage dependencies and builds; test with JUnit 5 and Mockito.

SYNTAX

```
mvn test
gradle build
```

EXAMPLE

```
@Test void adds() { assertEquals(5, calc.add(2, 3)); }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A kitchen supplier and a quality inspector: Maven/Gradle fetch libraries and run tests.

தமிழ்

மளிகை சப்ளையரும் தர ஆய்வாளரும்: Maven/Gradle நூலகங்களை எடுத்து tests இயக்குகின்றன.

PROS

- Dependency management
- Standard project layout
- CI friendly

CONS

- XML/Kotlin DSL learning
- Slow builds on big projects

ADVANCED / NEW FEATURES

- Gradle build cache
- JUnit 5/6
- Maven 4

Master Level

21. JVM Internals and Garbage Collection

DEFINITION

Class loading, heap/stack, GC algorithms (G1, ZGC), tuning and profiling with JFR/VisualVM.

SYNTAX

```
java -Xmx512m -XX:+UseG1GC App
```

EXAMPLE

```
java -Xmx512m -XX:+UseG1GC App
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A housekeeping team clears unused things; the JVM garbage collector frees memory automatically.

தமிழ்

பயன்படாதவற்றை வீட்டுப் பணியாளர்கள் அகற்றுவது போல JVM garbage collector நினைவகத்தை தானாக விடுவிக்கிறது.

PROS

- No manual memory management
- Mature GCs

CONS

- GC pauses if untuned
- Memory overhead

ADVANCED / NEW FEATURES

- ZGC generational (21)
- Compact object headers (24/25)
- AOT cache (24+)

22. Design Patterns and SOLID

DEFINITION

Singleton, Factory, Builder, Strategy, Observer; SOLID principles for maintainable code.

EXAMPLE

```
var order = Order.builder().id(1).customer("Priya").build();
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A well-known recipe (pattern) such as 'tiffin combo' that many hotels reuse; design patterns are reusable solutions.

தமிழ்

பல உணவகங்கள் பயன்படுத்தும் 'டிபன் காம்போ' செய்முறை போல design patterns மீண்டும் பயன்படுத்தும் தீர்வுகள்.

PROS

- Shared vocabulary
- Proven solutions

CONS

- Over-engineering
- Unneeded abstraction

ADVANCED / NEW FEATURES

- Records and sealed types simplify many patterns

23. Real Projects

DEFINITION

Library management (console + JDBC), banking system, REST API with Spring Boot, multi-threaded downloader.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A library management system, bank app and REST API are the standard practice projects that combine all topics.

தமிழ்

நூலக மேலாண்மை, வங்கிச் செயலி, REST API ஆகியவை எல்லா தலைப்புகளையும் இணைக்கும் நிலையான பயிற்சி project-கள்.

PROS

- Real-world practice

CONS

- Time consuming

ADVANCED / NEW FEATURES

- Try Java 25 LTS and Spring Boot 4

24. Best Practices and Interview Prep

DEFINITION

Immutability, meaningful names, avoid null, favour composition, `equals/hashCode`, collections complexity, concurrency basics.

EXAMPLE

```
@Override public boolean equals(Object o) { /* ... */ }
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A driving test before issuing a licence; interview prep checks basics like OOP, collections and concurrency.

தமிழ்

உரிமம் வழங்கும் முன் ஓட்டுநர் தேர்வு போல interview தயாரிப்பு OOP, collections, concurrency அடிப்படைகளைச் சோதிக்கிறது.

PROS

- Confidence in interviews

CONS

- Memorising without practice does not help

ADVANCED / NEW FEATURES

- Modern Java features appear in newer interviews

Interview Question Bank

Q1. JDK vs JRE vs JVM?
JVM runs bytecode; JRE = JVM + libraries; JDK = JRE + development tools (javac).

Q2. `==` vs `equals()`?
`==` compares references (or primitives); `equals()` compares logical content.

Q3. Why is String immutable?
Security, caching in the string pool, thread safety and hash code stability.

Q4. ArrayList vs LinkedList?
ArrayList has fast random access; LinkedList is faster for inserts/removals at ends or via iterator.

Q5. How does HashMap work?
Hashes keys to buckets; collisions use linked lists/trees; resizes when load factor is exceeded.

Q6. Abstract class vs interface?
Abstract classes share state and code; interfaces define contracts (with default methods) and allow multiple implementation.

Q7. Checked vs unchecked exceptions?
Checked must be declared or caught; unchecked (RuntimeException) need not.

Q8. What is garbage collection?
Automatic reclamation of memory from unreachable objects.

Q9. `synchronized` vs `volatile`?
`synchronized` gives mutual exclusion and visibility; `volatile` only guarantees visibility.

Q10. What is a functional interface?
An interface with exactly one abstract method, usable with lambdas.

Capstone Project Ideas

1. Student management (console)
2. Banking application with exceptions
3. Library system with JDBC
4. Multithreaded file downloader
5. REST API with Spring Boot